

ASSIGNMENT - 10.1

Name : CH.Charan Kumar

Roll: 2303A54006

Batch : 47 A

Task 1 – Syntax and Logic Errors

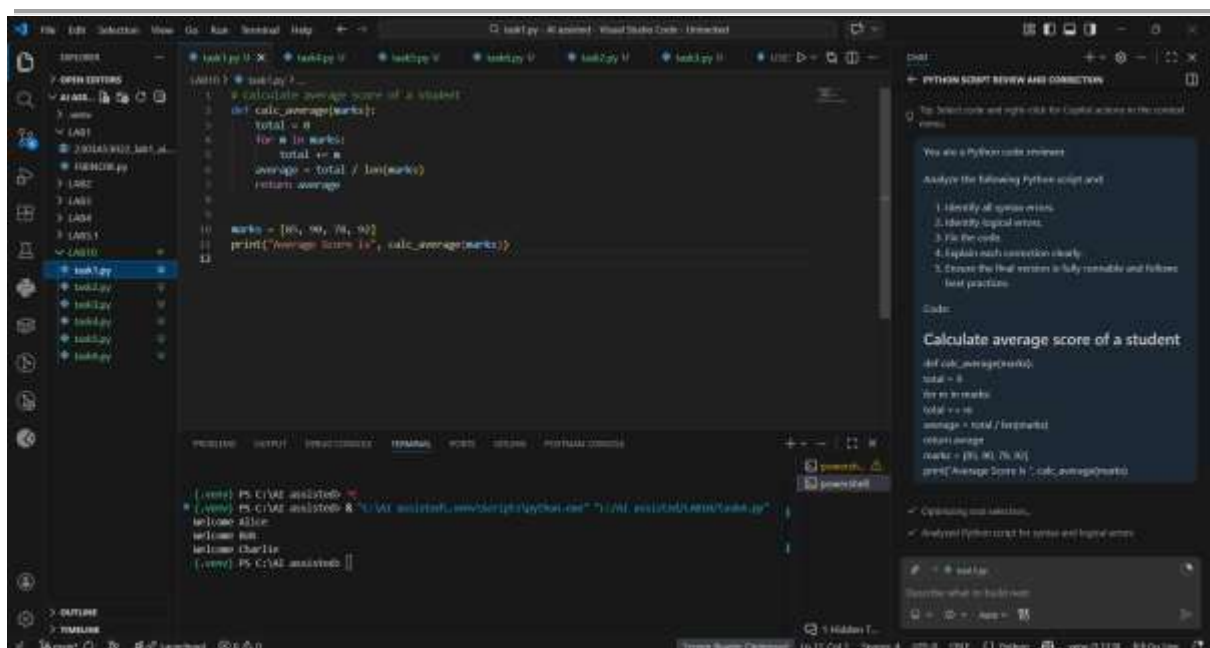
You are a Python code reviewer.

Identify syntax and logical errors in the following code.

Fix the errors and provide corrected runnable code with explanation.

Code:

```
# Calculate average score of a student def
calc_average(marks):
total = 0 for m
in marks:
total += m average = total /
len(marks) return avrage
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```



Explanation

- Fixed indentation errors.
 - Corrected typo avrage → average.
 - Closed missing parenthesis in print statement.
 - Added empty list check to avoid ZeroDivisionError.
 - Added docstring for better documentation.
-

Task 2 – PEP 8 Compliance

Task Prompt Used

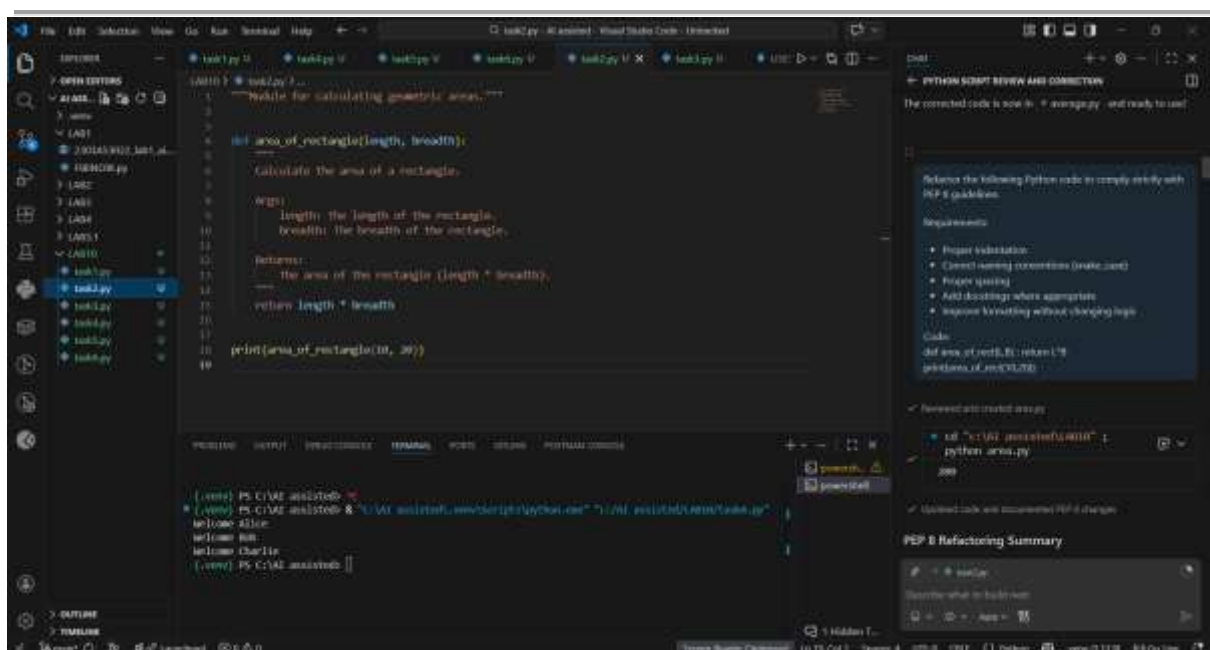
Refactor the following code to strictly follow PEP 8 guidelines.

Improve formatting and naming without changing logic.

Code:

```
def area_of_rect(L,B) : return L*B
```

```
print(area_of_rect(10,20))
```



Explanation

- Used snake_case naming.
- Proper indentation and spacing.
- Added docstring.
- Improved readability.

Task 3 – Readability Enhancement

Task Prompt Used

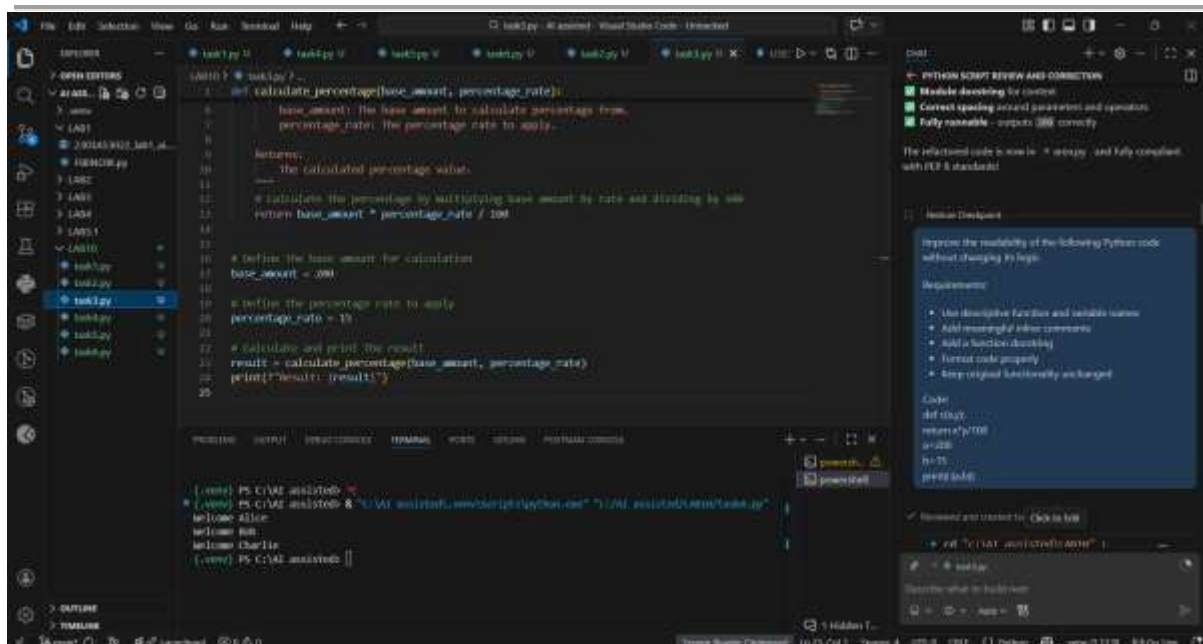
Improve readability of the following code.

Use descriptive names, comments, and proper formatting.

Do not change logic.

Code:

```
def c(x,y):  
    return x*y/100  
a=200 b=15  
print(c(a,b))
```



Explanation

- Function renamed for clarity.
- Meaningful variable names used.
- Added documentation.
- Proper formatting applied.

Task 4 – Refactoring for Maintainability

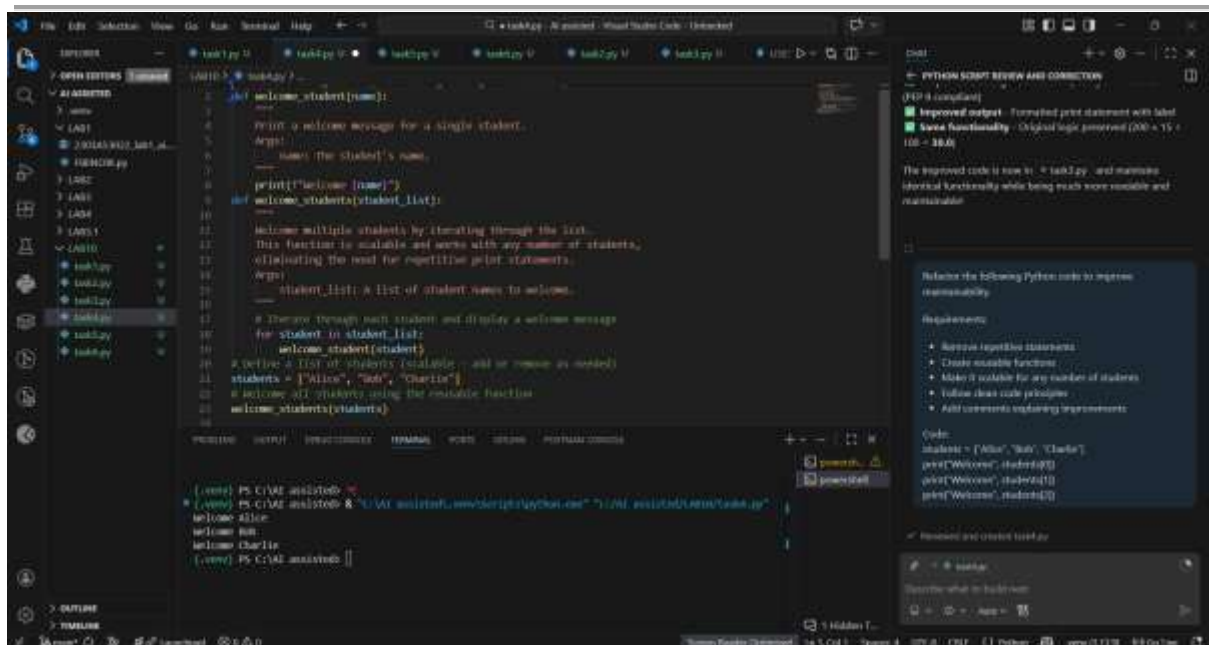
Task Prompt Used

Refactor the following code to remove repetition.

Create reusable function and improve maintainability.

Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0]) print("Welcome",
students[1]) print("Welcome", students[2])
```



Explanation

- Removed repetitive statements.
- Created reusable function.
- Code is scalable.

- Improved maintainability.

Task 5 – Performance Optimization

Task Prompt Used

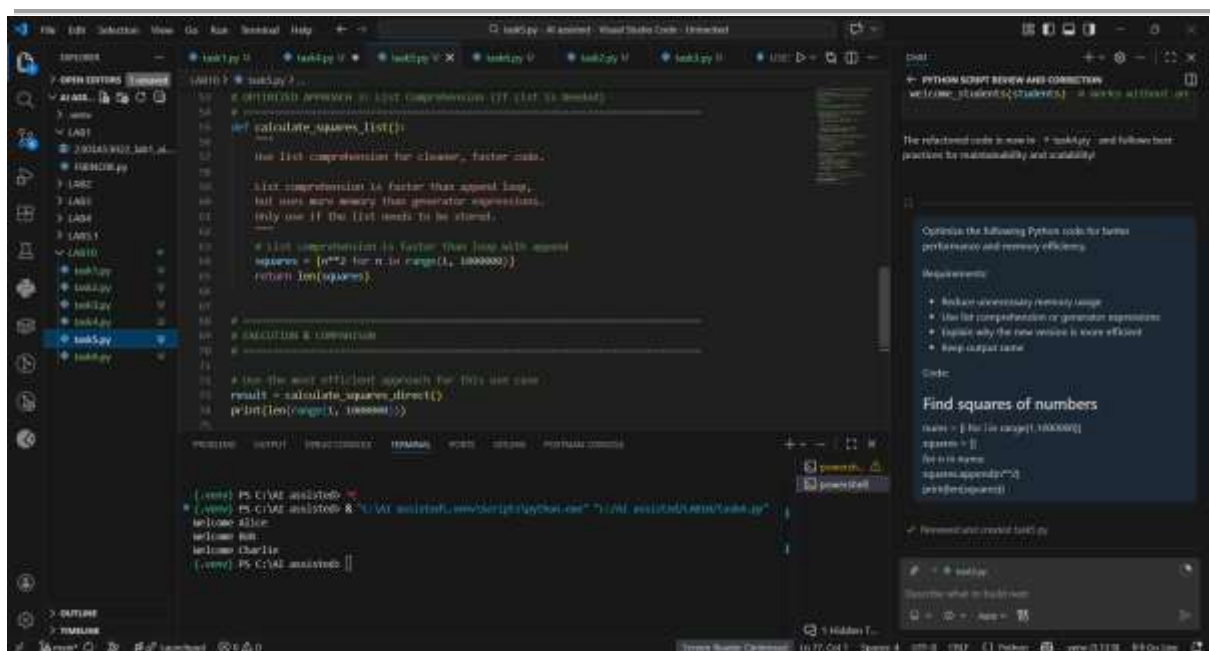
Optimize the following Python code for performance.

Use list comprehension or generator expressions.

Keep output same.

Code:

```
nums = [i for i in range(1,1000000)]  
  
squares = []  
for n in nums:  
    squares.append(n**2)  
print(len(squares))
```



Explanation

- Removed unnecessary list creation.
 - Used list comprehension (faster).
 - Reduced memory usage.
 - More Pythonic approach.
-

Task 6 – Complexity Reduction

Task Prompt Used

Simplify the following nested if-else logic.

Reduce complexity while maintaining same functionality.

Code:

```
def grade(score):
```

```
    if score >= 90:
```

```
        return "A" else:
```

```
    if score >= 80:
```

```
        return "B"
```

```
    else:
```

```
        if score >= 70:
```

```
            return "C"
```

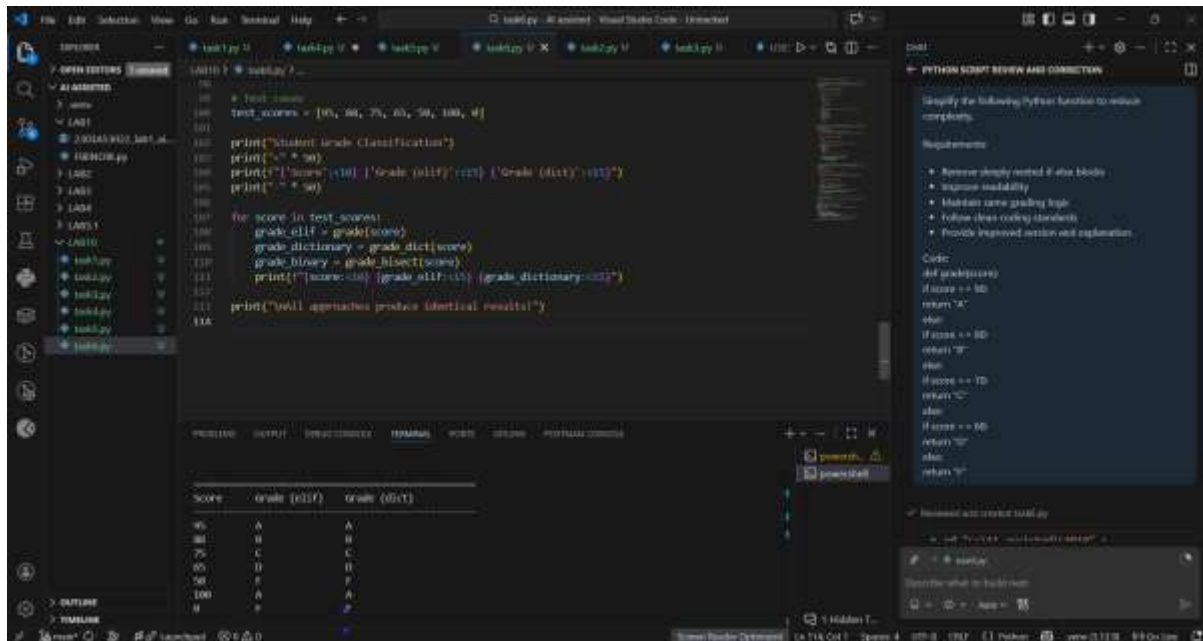
```
        else: if score
```

```
            >= 60: return
```

```
            "D"
```

```
    else:
```

return "F"



```
100 # test scores
101 test_scores = [95, 88, 75, 65, 50, 100, 0]
102
103 print("Student Grade Classification")
104 print("\n" * 50)
105 print(f"{'score':<10} | {'Grade (elif)':<15} | {'Grade (dict)':<15}")
106 print("\n" * 50)
107
108 for score in test_scores:
109     grade_elif = grade(score)
110     grade_dictionary = grade_dict(score)
111     grade_dictionary = grade_dict(score)
112     print(f"{'score':<10} | {'Grade (elif)':<15} | {'Grade (dict)':<15}")
113
114 print("Both approaches produce identical results!")
```

Score	grade (elif)	grade (dict)
95	A	A
88	B	B
75	C	C
65	D	D
50	F	F
100	A	A
0	F	F

Explanation

- Removed deeply nested conditions.
- Used elif for clarity.
- Improved readability and maintainability.